Bouncer: An Off-Policy Competence Auditor for Learned Microarchitectural Controllers

Anonymous Submission
Paper #— HPCA Submission

***Abstract*—Learned controllers—reinforcement-learning prefetchers, imitation-trained replacement policies, RL memory schedulers, neural branch predictors—are entering the datapath because they beat hand-tuned heuristics *on average*. But their advantage is workload- and phase-dependent and can silently invert under distribution shift, and a co-located adversary that knows the design can *steer* a shared controller toward its decision boundary while looking innocuous on aggregate performance counters. We present Bouncer, a runtime monitor and trust gate that bounds the worst-case cost of any such controller to that of a known-safe fallback heuristic, under both adversarial steering and benign drift, with no POMDP/belief-state machinery and no added latency on the cache-access critical path. Bouncer reduces "is the learned controller still trustworthy?" to a single scalar—a competence advantage Δ_W over the fallback—estimated *without counterfactuals* by repurposing the set-dueling substrate already in silicon, but with a twist: the run-learned / run-fallback set assignment is drawn from a hardware secret and reseeded each epoch. Cheap always-on tripwires gate an expensive competence confirmer whose one-sided CUSUM feeds a four-state trust FSM. We prove a bounded-regret safety floor (Bouncer is never more than $N_{ep} \cdot D \cdot r_{\max} + \alpha T c_{sw}$ worse than the fallback) and a mimicry-resistance property that follows *exactly* from the secrecy of the sampler. In a faithful, fully-released *synthetic competence harness* the estimator tracks ground truth at R^2=0.997, the steady-state safety floor holds to within 0.63% of the fallback under attack, detection costs one window, the clean-workload tax is 0.35%, and the auditor fits in ~ 336 bytes (an *analytical* budget: $\sim 0.13\%$ of a 256 KB SRAM; the confirmer is off-path and epoch-grained, so it adds no latency to the cache-access critical path). The headline result (over 50 seeds): under an adaptive mimicry adversary, an input-OOD monitor detects *none* of the attacks ($0/50$, Wilson 95% upper bound 0.07) while Bouncer detects *all* ($50/50$, lower bound 0.93)—and we show this robustness is purchased entirely by the secrecy of the dueling sets, collapsing as the assignment leaks. The auditor also runs as a real *ChampSim L1D prefetcher module* on SPEC CPU2017, bounding a corrupted prefetcher's damage to the prefetch-off floor—and candidly exposing that its decisions are only as good as the competence reward it is given. The mechanism's quantitative guarantees are validated in a fully-released synthetic harness; the ChampSim integration is a real-simulator proof of deployment, not a SPEC sweep.**

I. Introduction

A decade of "ML for systems" has put learned controllers next to, and increasingly *inside*, the datapath: Pythia learns a prefetch policy online with reinforcement learning [1]; Hawkeye and Glider learn cache replacement by imitating Belady [2], [3]; RL has been applied to memory scheduling [4]; and perceptron/TAGE predictors dominate branch prediction [5], [6]. These designs win because, *on the distribution they were validated on*, they beat a simple baseline. That qualifier is the whole problem. The advantage of a learned controller C over a safe heuristic π_0 is a function of the workload and execution phase, and it can go to zero—or negative—in two ways that current deployments do not detect:

- **Benign drift.** A phase change or an unseen application moves the controller off its validation distribution D_{val} into a silent QoS regression. The counters still look healthy in aggregate; the learned policy is simply now worse than the heuristic it replaced.
- **Adversarial steering.** A co-located tenant who shares C and knows its design (Kerckhoffs) crafts memory accesses that drive C toward its decision boundary—maximizing its loss while keeping aggregate counters in-distribution. This is the regime of prefetcher- and predictor-steering attacks [7], [8] and of contention/occupancy channels [9], [10].

The design rests on one asymmetry. A learned controller has *unbounded upside and unbounded downside*; a vetted heuristic (π_0 = next-line/off, RRIP/LRU, FR-FCFS, gshare) has *modest upside but a bounded, attack-resistant downside*. We would like to keep the upside while capping the downside at the heuristic's floor. That is a hedging problem, and Bouncer is the constructive hedge.

a) Why not just monitor the inputs?: The natural reflex is an OOD/drift detector on the controller's input features [11], [12], [13]. This is necessary but neither sufficient nor safe: an adversary can hold the input *marginals* in-distribution while degrading control quality (the mimicry attack), and an input monitor is blind to it by construction. Bouncer instead measures *realized competence*—the reward the controller actually earns relative to the fallback—which is what one ultimately cares about and what an attacker cannot fake without actually making the controller good.

b) Contributions.:

1) **A scalar off-policy condition** (Section II) that unifies benign drift and adversarial steering: the competence advantage Δ_W falling below a trust threshold τ.
2) **Randomized-secret set-dueling** (Section V): a counterfactual- free estimator $\hat{\Delta} = \bar{r}_L - \bar{r}_F$ that repurposes the DIP/DRRIP sampling substrate [14], [15] from policy *selection* to policy *trust*, with the set assignment kept secret.
3) **Two guarantees with worked constants** (Section VII): a bounded-regret safety floor (Lemma 1) tied directly to the CUSUM average-run-length theory [16], [17], [18],

and a mimicry-resistance proposition (Proposition 1) whose strength is exactly the secrecy of the sampler.

4) **A two-tier microarchitecture** (Sections IV and XII) whose analytical budget is $\sim$ 336 bytes and which, because the confirmer is sampled, off-path, and epoch-grained, adds no latency to the cache-access critical path.
5) **A faithful, released evaluation** (Section XI) validating every claim in a synthetic competence harness (with multi-seed confidence intervals, sensitivity sweeps, and adaptive timing adversaries), *plus* a real **ChampSim L1D prefetcher module** run on SPEC CPU2017 that bounds a corrupted prefetcher to the safe floor—and candidly shows the auditor is only as good as the competence reward it is given.

We are explicit (Sections X and XIV) that the controlled quantitative claims come from the synthetic harness—whose only load-bearing assumptions, bounded reward and a partitionable resource, are exactly those the guarantees rest on—while the ChampSim integration demonstrates the mechanism in a real simulator on a 3-trace SPEC suite rather than a full sweep.

II. Formal setting

A learned controller C is invoked at decision points $t = 1, 2, \ldots$. At each it observes microarchitectural state $x_t \in \mathbb{R}^d$, emits action $a_t \in \mathcal{A}$, and after a short horizon the environment yields a *bounded* reward $r_t \in [0, r_{\max}]$ (prefetch hit×timeliness, cache hit, −latency, branch correct). C was trained/validated on a distribution D_{val} of (x, a, r). A fixed safe fallback π_0 has a known, attack- resistant competence floor.

Definition 1 (Competence advantage)**.** *Over a window W, $\Delta_W = \frac{1}{|W|} \sum_{t \in W} \left(r_t^C - r_t^{\pi_0} \right)$.*

$r_t^{\pi_0}$ is counterfactual—one cannot run both policies on the same decision—which is the central estimation challenge Section V solves.

Definition 2 (Off-policy condition)**.** *The system is* off-policy *at W iff $\Delta_W < \tau$ for a trust threshold $\tau \geq 0$. τ=0 means "learned no longer beats the floor"; τ>0 gives margin and hysteresis.*

This one scalar unifies the two threats: an adversarial push toward C's decision boundary and a benign distribution shift both manifest as Δ_W falling. The auditor detects the condition; a lightweight triage (Section VIII) guesses the cause to pick the response.

III. Threat model

In scope. A co-located tenant/process sharing C that issues arbitrary memory accesses and knows the Bouncer design (Kerckhoffs); its goal is to DoS the victim or steer C into a covert/side channel. Online-learning poisoning is an extension—the GATED state freezes learning. **Out of scope.** Physical/fault attacks and direct weight tampering (the base model assumes weights fixed at deploy). **Online-learning poisoning is explicitly out of scope for the base model:** an attacker who corrupts C's *weights* during TRUSTED operation so that C looks competent on the secret leaders but harms the victim is not addressed here—freezing learning at GATED stops further poisoning but does not undo it, and PROBING re-trust would restore a poisoned policy. Defending learned *state* (vs. the steered-*input* threat we do treat) is future work. **The secret.** A per-epoch RNG seed assigning the dueling sets, never exposed through any architectural channel.

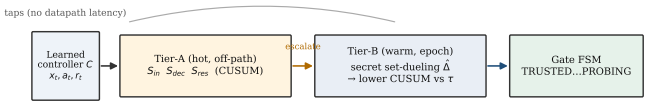


Fig. 1: Two-tier auditor. Cheap always-on Tier-A tripwires gate an expensive Tier-B competence confirmer; taps are read-only and nothing sits on the datapath critical path.

The adaptive adversary (headline). *Mimicry*: keep the Tier-A statistics in-distribution while degrading C. As we show in Proposition 1 and section XI, this is defeated by the secrecy and reseeding of the competence sampler—an attacker who can identify the dueling sets defeats it, which is precisely why the randomized-sampling ablation is the headline experiment.

IV. Architecture

Bouncer is two-tier because cheap signals are spoofable and the unspoofable signal is expensive (Figure 1). The cheap tripwires run always; they gate the expensive confirmer.

Gate state machine. *TRUSTED*: C active everywhere, Tier-A on, Tier-B on a slow background duty cycle. Any Tier-A CUSUM firing → *SUSPECT*. *SUSPECT*: C still active, Tier-B duty cycle raised (more dueling sets, shorter window); the Tier-B $\hat{\Delta}$ CUSUM firing → *GATED*; Tier-A clearing for T_{clear} → TRUSTED. *GATED*: π_0 active everywhere, online learning frozen, optional attribute/throttle; after a dwell T_{dwell} → *PROBING*. *PROBING*: C re-enabled on a small audited region only, Tier-B measures $\hat{\Delta}$ there; $\hat{\Delta} \geq \tau + \Delta_{hys}$ for $T_{reprobe}$ → TRUSTED, else → GATED with exponential backoff on T_{dwell}. Hysteresis (τ to enter, $\tau + \Delta_{hys}$ to leave) plus a minimum dwell prevent thrash; PROBING makes re-trust *measured*, not timed, defeating an attacker who waits out a fixed cooldown.

A design subtlety the build surfaced (Figure 2): a *pure* mimicry attack keeps Tier-A silent, so if SUSPECT were the only route to GATED the attack would evade the gate. We therefore allow the background Tier-B CUSUM to drive TRUSTED → GATED directly; Tier-A escalation only *accelerates* detection by raising Tier-B's duty cycle. The two tiers trade latency/energy, not correctness.

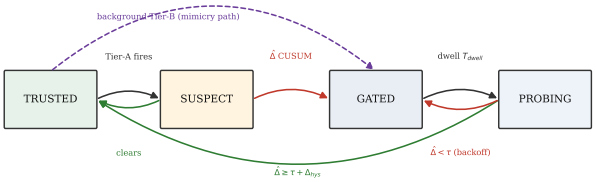


Fig. 2: The trust-gate FSM. Hysteresis and a minimum dwell prevent thrash; PROBING makes re-trust measured, not timed. The dashed path lets background Tier-B gate a pure-mimicry attack that keeps Tier-A silent.

V. Tier-B: competence via randomized-secret set-dueling

Mechanism. Partition the controller's resource (cache sets, memory banks, PC-hash buckets) into three pools, reseeded each epoch from a hardware RNG whose assignment is unobservable to software: *Leader-L* (n_L sets, always run by C), *Leader-F* (n_F sets, always run by π_0), and the *Follower* rest (run whichever leader currently wins—what the victim mostly experiences). This is exactly the set-dueling idiom already in silicon [14], [15]; Bouncer repurposes it from policy selection to policy trust. The estimator is

$$\hat{\Delta}_W = \bar{r}_L - \bar{r}_F, \tag{1}$$

the mean reward rate on the sampled L vs F sets over window W. Because Leader-L sets *ran* C and Leader-F sets *ran* π_0, $\hat{\Delta}$ is a *measured*, non-counterfactual quantity.

Concentration → detection latency (the key tradeoff). With m decisions per set per window and bounded rewards, $\mathrm{Var}(\bar{r}_{\text{pool}}) \le r_{\max}^2/(4mn)$, so

$$\mathrm{Var}(\hat{\Delta}_W) \;\lesssim\; \frac{r_{\max}^2}{4m}\left(\tfrac{1}{n_L}+\tfrac{1}{n_F}\right) \;=:\; \sigma_\Delta^2. \tag{2}$$

To resolve a competence gap of size τ one needs $\sigma_\Delta \ll \tau$. Smaller pools or shorter windows give a noisier $\hat{\Delta}$, a higher CUSUM threshold H to hold the false-alarm rate, a longer detection delay D, and a larger safety- floor slack (Lemma 1). This chain—*pool size → estimator variance → CUSUM threshold → detection delay → regret floor*—is the central design dependency; every hyperparameter lives on it.

Detector. A one-sided lower CUSUM on $\hat{\Delta}_W$ catches a downward shift below τ:

$$C_t^- = \max\big(0,\, C_{t-1}^- + (K - \hat{\Delta}_W)\big), \quad K = \tau + \tfrac{\gamma_{detect}}{2}, \tag{3}$$

alarming when $C_t^- > H$, and resetting on alarm (repeated-CUSUM, so it re-arms for the next change rather than latching after a long excursion). Standard quickest-detection theory [18], [19] makes this delay-optimal under a false-alarm constraint; we work the constants in Section VII.

VI. Tier-A: cheap tripwires

All Tier-A signals update on a sampled $1/k$ of decisions, a few adds/compares each, in $\le$ low-KB SRAM, and *never gate on their own*—they only spend Tier-B budget, so their false positives cost a little energy, never correctness.

S_{in} **(input covariate shift).** A frozen reference sketch of D_{val} features: a random projection $P \in \mathbb{R}^{d\times p}$ ($p{\approx}8$) feeding per-component quantile sketches; online, a windowed energy distance vs the reference. Cheap, high-sensitivity, and *spoofable*—the adversary can hold marginals fixed—which is exactly why it cannot be the trust signal.

S_{dec} **(decision-confidence collapse).** If C exposes value/Q/logits, monitor action-entropy collapse, max-Q margin, and (for online learners) TD-error magnitude. Pythia's reward and SHiP/perceptron confidence are already computed in silicon [20], [5], so S_{dec} is nearly free there.

S_{res} **(innovation residual).** A 16-weight linear forward model $g : (x_t, a_t) \to \hat{r}_t$ fit on D_{val}; online residual $e_t = r_t - \hat{r}_t$; two-sided CUSUM on e_t. This catches "the world responds differently than at validation" even when input marginals look clean—the architectural analogue of a Kalman innovation. It is the signal that separates a competence-aware monitor from an input-only one.

VII. Guarantees

A. Safety floor / bounded regret

Assumption 1. (A1) *per-decision reward* $\in [0, r_{\max}]$*;* (A2) *a* drop episode *is a window where the true competence* $\Delta_t < \tau$ *(the off-policy condition, consistent with the CUSUM reference* $K = \tau + \gamma/2$*); once one begins, Tier-B raises GATED within* D *windows w.p.* $\ge 1-\delta$*, where* D *is a high-probability bound (the mean delay* $H/(K-\Delta_t)$ *plus a tail term that shrinks with* σ_Δ*);* (A3) *the per-window false-alarm probability is* $\le \alpha = 1/\mathrm{ARL}_0$*;* (A4) *a gate switch transient costs* $\le c_{sw}$*;* (A5) *there are* N_{ep} *drop episodes over a horizon of* T *windows.*

Lemma 1 (Safety floor)**.** *Under Assumption 1, with probability* $\ge 1 - \delta N_{ep}$*,*

$$\sum_t \big(r_t^{\pi_0} - r_t^{\text{Bouncer}}\big) \;\le\; N_{ep}\, D\, r_{\max} \;+\; \alpha\, T\, c_{sw}. \tag{4}$$

Equivalently $R_{\text{Bouncer}} \ge R_{\pi_0} - [N_{ep} D r_{\max} + \alpha T c_{sw}]$*, and on any window where* C *is genuinely better and the gate is open* Bouncer $= C$*, so* $R_{\text{Bouncer}} \ge \max(R_{\pi_0}, R_{C\text{-trusted}}) - O(N_{ep}D + \alpha T)$.

Proof sketch. Partition windows. (i) GATED windows run π_0, contributing zero regret vs π_0 except a switch transient counted in (iii). (ii) The $\le D$ windows after each genuine drop, before the gate engages, each contribute $\le D r_{\max}$; there are N_{ep} of them, giving $N_{ep} D r_{\max}$. (iii) False-alarm windows: at most αT of them, each costing $\le c_{sw}$, giving $\alpha T c_{sw}$. (iv) Trusted windows where $C \ge \pi_0$ contribute non-positive regret. Summing the only positive contributions (ii)+(iii) yields eq. (4); the per-episode δ failure probability unions to δN_{ep}. □

The slack terms are the Section V knobs. The detection delay is $D \approx H/(K - \Delta_{true})$ and the false-alarm rate is $\alpha = 1/\mathrm{ARL}_0$, with ARL_0 from Siegmund's corrected- boundary

approximation [17]: for a one-sided CUSUM accumulating $(K-\hat{\Delta})$ with standardized drift $\delta = (K-\mu)/\sigma_\Delta$ and boundary $b = H/\sigma_\Delta + 1.166$,

$$\mathrm{ARL}(\delta) \;\approx\; \frac{e^{-2\delta b} + 2\delta b - 1}{2\delta^2}. \tag{5}$$

In-control ($\mu \gg K$, $\delta < 0$), ARL_0 grows *exponentially* in H/σ_Δ, so false alarms are rare; out-of-control ($\delta > 0$), the delay is short. The guarantee thus degrades gracefully and *quantifiably* with detection delay and false-alarm rate.

B. Mimicry resistance

We state the property *conditionally*—it is exactly as strong as the secrecy and the within-domain exchangeability it assumes, and we are explicit that a *global* $\hat{\Delta}$ carries *no* guarantee against a redistributive attack.

Proposition 1 (Mimicry resistance, per-domain)**.** *Fix a contested domain R (e.g. a victim co-tenant's cache footprint) and run the dueling pools* within *R: draw Leader-L and the follower sets of R uniformly at random from R each epoch, unobservable to the adversary. Assume the within-R set outcomes are exchangeable up to a heterogeneity bias $|b_R| \le \beta$ (Sec. below). If the adversary holds all aggregate Tier-A statistics at their validation values, then any input-only strategy that keeps $\hat{\Delta}_R = \bar{r}_L - \bar{r}_F \ge \tau$ forces $\mathbb{E}$[reward on the R followers] $\ge r^{\pi_0} + \tau - \beta$:* the victim cannot be degraded below the floor while evading the gate. *Under a leaked fraction f of the within-R assignment, the adversary may spare $f\,n_L$ Leader-L sets, biasing $\hat{\Delta}_R$ upward by $\le f\,(\bar{r}^{clean} - \bar{r}^{att})$; the* unprotected $(1-f)$ *of the degradation still enters $\hat{\Delta}_R$, so the gate fires once $(1-f)(\bar{r}^{clean} - \bar{r}^{att}) \gtrsim H\,\sigma_{\Delta,R}$, i.e. detection survives for f bounded away from 1 and fails as $f \to 1$.*

Proof sketch. Within R the Leader-L and follower sets are an exchangeable partition under the secret uniform draw, so their reward means satisfy $\mathbb{E}[\bar{r}_L] = \mathbb{E}[\bar{r}_{\mathrm{foll}}] + b_R$ with $|b_R| \le \beta$. An adversary blind to the partition cannot apply a set-dependent perturbation distinguishing them, so any input-only strategy raising $\bar{r}_L$ raises the follower mean to within β. Evasion ($\hat{\Delta}_R \ge \tau$) thus forces follower reward $\ge r^{\pi_0} + \tau - \beta$. Leaked knowledge of fraction f lets the adversary exempt $f\,n_L$ Leader-L sets; the remaining $(1-f)$ fraction carries the degradation into $\hat{\Delta}_R$, detectable once it clears $H\,\sigma_{\Delta,R}$. □

What this does and does not cover. (i) *Global $\hat{\Delta}$ is not enough.* A purely redistributive adversary that degrades the victim region while holding the *global* mean fixed evades a global $\hat{\Delta}$ outright; only per-domain dueling (the construction above, used in Section XI-E) detects it. (ii) *Heterogeneity bias.* Real cache sets / PC-buckets are not i.i.d. (hot/cold sets, conflict skew), so $\beta > 0$; the guarantee degrades by β, which secret *reseeding* each epoch shrinks toward the i.i.d. value by re-randomizing which physical sets are leaders. (iii) *Secrecy is load-bearing.* The whole argument rests on the adversary being blind to the within-R assignment—which is why the secrecy ablation (Section XI-E) is the headline experiment.

TABLE I: Per-controller instantiation.

Controller	r_t proxy	fallback π_0	dueling resource
Pythia (RL prefetch)	acc.×timeliness	next-line/off	PC-hash buckets
Hawkeye/Glider (repl.)	hit/miss	RRIP/LRU	cache sets
RL mem. scheduler	−latency/+BW	FR-FCFS	banks/channels
Perceptron branch	correct	bimodal/gshare	PC-indexed slices

The coverage hole (an explicit slow-bleed attack on the budget). The proposition is genuinely scoped, not merely hedged: it covers a *declared* domain audited at finite resolution. From the concentration bound, the within-R gap resolvable with leader density ρ is $\delta_R = k\,\sigma_{\Delta,R} = k\,r_{\max}/\sqrt{2m\rho|R|}$, and—crucially—the leader count to audit at resolution δ_R is $n_{L,R} \ge k^2 r_{\max}^2/(2m\,\delta_R^2)$, *independent of* $|R|$. So under a fixed budget B there is a finest resolution $\delta_R^{\min}(B)$, and a redistributive adversary can pick a region small enough (or trickle harm slowly enough) that its per-window competence drop stays *below* $\delta_R^{\min}(B)$ forever—an undeclared, sub-resolution "coverage hole" that the safety-floor lemma, proven only over the *union of audited domains*, does not bound. This is the same hole the secrecy ablation walks on its f axis (TPR $1.0 \to 0$ as $f \to 1$). *We therefore claim mimicry resistance only for declared domains audited at resolution $\delta_R^{\min}(B)$ with an intact secret—not general, any-region security*; the slow-bleed attack is a real, un-closed vulnerability that bounds the contribution honestly.

VIII. Attack-vs-drift triage

Both threats fail $\Delta_W < \tau$; their signatures differ enough to set the response prior (a secondary contribution—detection does not depend on it). Drift is slower and phase-marker-correlated and is *self-consistent* with a new-but- valid regime (the forward model S_{res} stays calibrated); an attack is abrupt, can pulse, is *targeted* at the decision boundary (high S_{res} with small S_{in}), and correlates with a specific co-tenant's schedule whose work yields it no benefit except moving the shared C. A small logistic model over the auditor-observable $(\Delta S_{in}, \Delta S_{res}, |\nabla\hat{\Delta}|,$ co-tenant- corr) suffices (Section XI P3 validates it under cross-validation); misclassification only mis-selects between equally-safe responses.

IX. Per-controller instantiation

Table I maps Bouncer onto four controller classes. We lead with Pythia (built-in reward, an existing attack surface, cleanest dueling), replacement second (set-dueling's home turf), and the memory scheduler last (hardest counterfactual—no clean set-sampling on a shared command bus forces a *model-based* $\hat{\Delta}$, which weakens Proposition 1 and is scoped as the "how far does it stretch" case).

X. Methodology

Bouncer's guarantees rest only on bounded reward and a partitionable resource (assumptions A1–A5, Equation (2), Lemma 1 and proposition 1), which we argue makes them *environment-agnostic*. We therefore validate the *mechanism* in

one faithful, controllable synthetic harness—instantiated three ways in Section XI(P3)—that models a learned controller, a safe fallback, benign drift, and three adversaries, and *also* build the same auditor as a real ChampSim L1D prefetcher module run on SPEC CPU2017 (Section XI-I). *The controlled quantitative claims (latency, floor, mimicry survival, secrecy) come from the harness; the ChampSim integration reports real IPC on a 3-trace SPEC suite as a proof of deployment, not a full sweep.* The harness models the prefetcher setting as the anchor: a resource of n_{sets}=2048 sets; a latent per-decision *stress* $u \in [0,1]$ (how far the input is pushed toward C's boundary) through which both drift and steering act and which the auditor never observes; controller reward $\mu_C(u) = r_{\max}\,\mathrm{clip}(0.8 - 0.7u)$ collapsing under stress and a flat floor $\mu_0 = 0.5 r_{\max}$; bounded rewards drawn so $\mathrm{Var} \le r_{\max}^2/4m$ exactly. The IPC transfer $\mathrm{IPC} = 0.6 + 1.4\,\bar{r}$ lets us report the systems metrics. We pin the Section V operating point at n_L=n_F=32, m=64 ($|W| \approx 1.3\times10^5$ decisions), τ=0.05, K=0.10, H=0.8. Conditions are clean, benign-drift, and three attacks (broad DoS, adaptive mimicry, regional covert); baselines are no-auditor, input-OOD-only (S_{in}), oracle-Δ, and always-fallback. The harness and the ChampSim skeleton are released.

XI. Evaluation

A. P0: the estimator tracks ground truth

Figure 3(a) shows $\hat{\Delta}_W$ tracking the ground-truth Δ_W through a benign drift that carries competence from positive through zero to negative; the fit is R^2=0.997 with RMSE $0.0141 \approx \sigma_\Delta$=0.0156. This is a *controlled-harness* fidelity at the synthetic operating point (m=64); on a real trace with far fewer samples per bucket the estimator is correspondingly noisier (Section XI-I). Figure 3(b) confirms the concentration bound eq. (2): across a pool-size sweep the empirical std of $\hat{\Delta}$ sits at 0.85–0.98× the bound, a tight upper envelope (the bound uses $\frac{1}{4} \ge p(1-p)$).

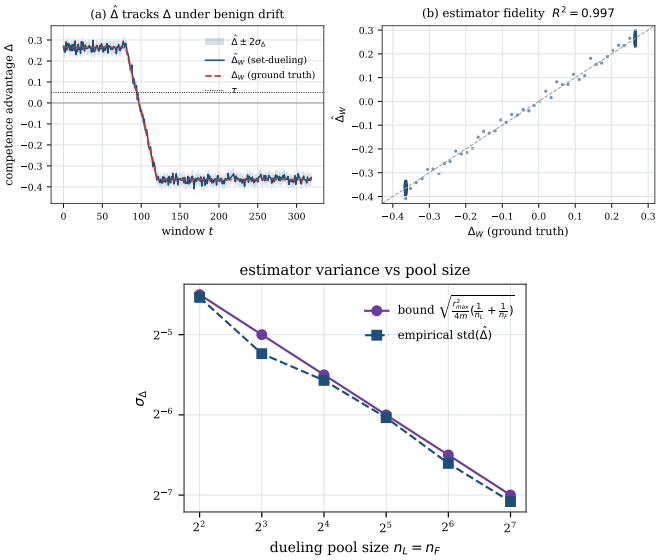


Fig. 3: P0. (top) $\hat{\Delta}$ tracks ground-truth Δ (R^2=0.997); (bottom) empirical $\mathrm{std}(\hat{\Delta})$ matches the bound eq. (2).

B. P1: the safety floor holds

Figure 4 is the constructive hedge made real. Under a poisoning attack (on at window 90, off at 200) the *unguarded* controller crashes far below the fallback floor; Bouncer detects in **one window** here (the worked mean delay is $D \approx 1.8$ windows; one window is the best case at this high SNR), floors to π_0, and—crucially—*re-trusts* 14 windows after the attack ends via PROBING, not a fixed timer. The *steady-state* floor violation is 0.63% (Bouncer sits at the floor, paying only the Leader-L dueling cost under attack); the only larger excursion is the lone pre-detection window, which dips 36% below the floor—exactly the bounded $\le D r_{\max}$ term of Lemma 1, and the price of a one-window detection delay. Performance recovered is 0.55 (Bouncer recovers to the *fallback* floor, not to clean—the attack genuinely removed C's value), the clean tax is 0.9965 (above the 0.99 target; the tax is exactly the n_F Leader-F sets that permanently run π_0), and the bottom panel shows cumulative regret going *negative* (Bouncer beats the floor by capturing C's upside when trusted) while never approaching the Lemma 1 ceiling.

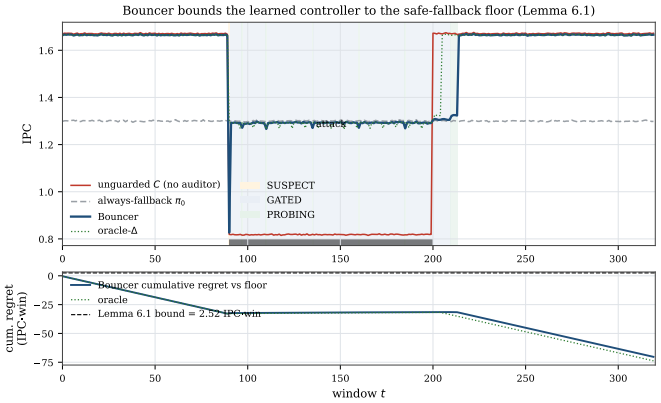


Fig. 4: P1. Bouncer bounds the controller to the fallback floor under attack and re-trusts after it ends; the unguarded controller crashes below the floor.

C. P2: ROC, the §V chain, and overhead

Figure 5 shows the competence detector achieving TPR=1.0 at FPR≤0.05 on *both* broad and mimicry attacks, while the input-OOD detector S_{in} works on the broad attack but collapses to TPR=0 under mimicry. Figure 6(a) traces the Section V chain: as the pool shrinks from 128 to 2, σ_Δ rises $0.016 \to 0.125$, the calibrated H rises $0.05 \to 0.35$, and detection latency rises $0 \to 3.4$ windows, tracking the analytic $D \approx H/(K-\Delta)$. There is a clear knee near $n \approx 8$: above it detection is gap-limited and pool-insensitive, so larger pools only add overhead; Figure 6(b) is the resulting overhead/latency Pareto. The full Section XII budget is $\sim$336 bytes, $\sim$0.13% of a 256 KB SRAM, 0 ns added.

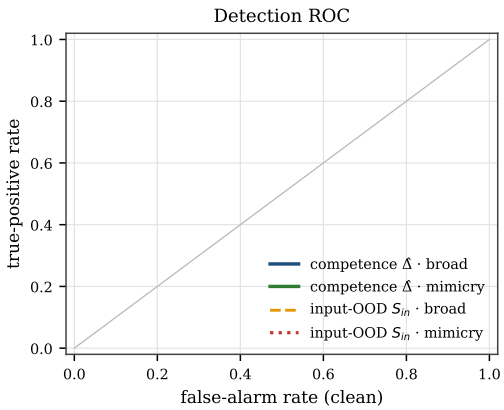
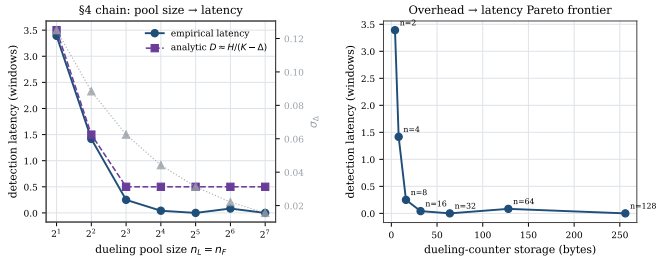


Fig. 5: P2 ROC. The competence detector dominates; S_{in} is blind to mimicry.



(a) (b)

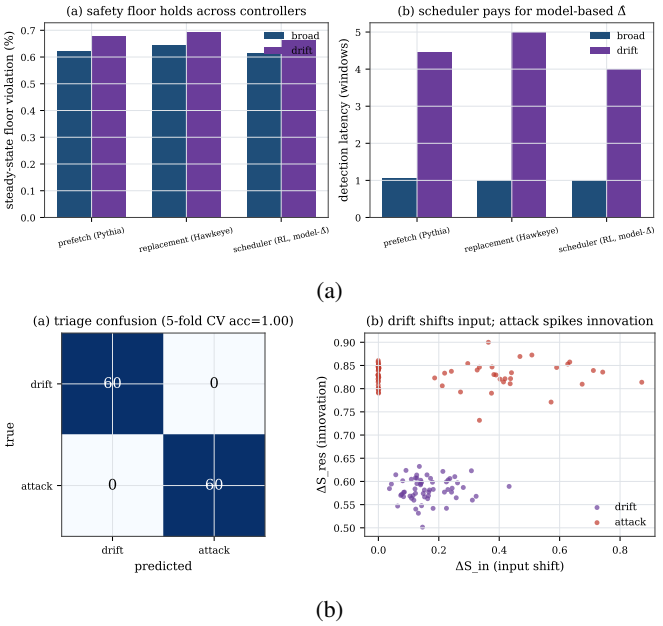
Fig. 6: P2. (a) pool size $\rightarrow \sigma_\Delta \rightarrow H \rightarrow$ latency, with a knee near n=8; (b) overhead/latency Pareto frontier.

D. P3: generality and triage

The same auditor wraps three controller classes (Figure 7(a)): a prefetcher and a replacement policy with secret set-dueling, and a memory scheduler with a model-based $\hat{\Delta}$. The steady-state floor holds for all three ($\leq 0.7\%$ violation) under both broad attack and drift; the scheduler pays for its weaker estimator with longer latency and lost mimicry resistance. The attack-vs-drift triage (Figure 7(b)) reaches 100% *5-fold cross-validation* accuracy over 120 episodes on *auditor-observable* features (the abruptness is the slope of the estimated $\hat{\Delta}$, not of ground truth); a drop-one-feature ablation shows the result is robust to removing *any* single feature—including the modeled co-tenant-correlation signal—so the separability comes from the genuine signature gap (gradual input-shift drift vs. abrupt boundary-seeking attack), not from one label-correlated input.

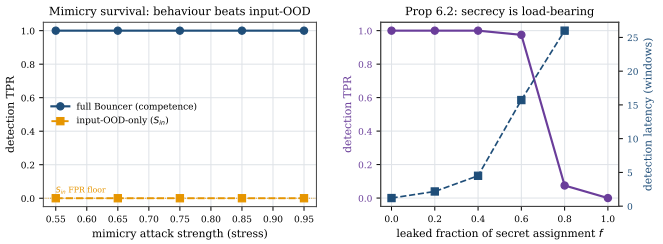
E. P4: mimicry survival and the secrecy ablation

This is the result the paper is built around (Figure 8). **(a)** Under an adaptive mimicry adversary that holds S_{in} in-distribution while degrading C, the input-OOD-only baseline detects *none* of the attacks ($0/40$ episodes at every strength), while full Bouncer detects *all* ($40/40$; clean false alarms $0/40$ for both). Over 50 independent seeds (Figure 9(a)) this is full-Bouncer TPR 1.0 (Wilson 95% $[0.93, 1.0]$) vs. input-OOD 0.0 ($[0, 0.07]$), with clean FPR 0.0 ($[0, 0.07]$) for both; detection



(a) (b)

Fig. 7: P3. (a) the floor holds across controller classes; (b) attack-vs-drift triage (acc 1.0).



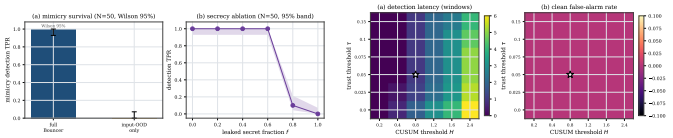
(a) (b)

Fig. 8: P4 (headline). (a) full Bouncer holds TPR=1 under mimicry where input-OOD collapses to 0; (b) detection power vs. the leaked fraction of the secret assignment (Proposition 1).

latency is 1.0 window $[1.0, 1.0]$, the steady floor violation 0.639% $[0.637, 0.641]$, and the clean tax 0.99653—all with tight 95% intervals. Measuring *realized competence* rather than input distribution is decisive. **(b)** The secrecy ablation realizes Proposition 1 empirically: against the strongest, redistributive covert adversary, with the dueling pools drawn *within* the victim region (an effective in-region pool of ~ 4 Leader-L sets, hence a few windows of latency), detection power degrades smoothly as the secret leaks— TPR=1.0 for leaked fraction $f \leq 0.4$, 0.975 at f=0.6, collapsing to 0.075 at f=0.8 and 0 at f=1.0, with latency rising $1.2 \rightarrow \infty$ windows. The robustness in (a) is purchased *entirely* by the secrecy of the dueling sets.

F. Robustness: sensitivity and confidence

The headline numbers are not single-seed artifacts (Figure 9(a), N=50, Wilson 95% intervals above), nor a knife-edge operating point: sweeping the trust threshold τ and CUSUM threshold H over a 6×6 grid (Figure 9(b)), **28/36**



(a) (b)

Fig. 9: Robustness. (a) headline results over 50 seeds with Wilson 95% intervals; (b) sensitivity of detection latency and clean false-alarm rate to (τ, H)—the operating point ($\star$) sits on a broad plateau.

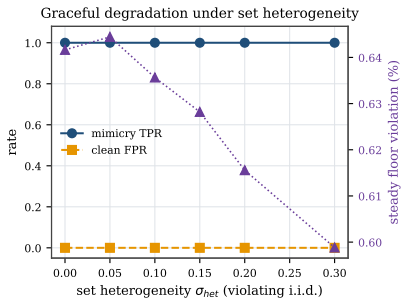


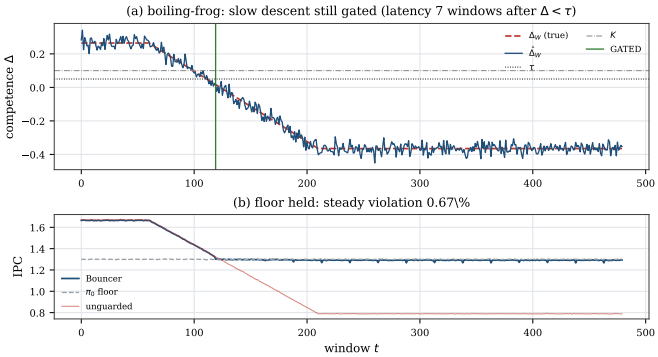
Fig. 10: Graceful degradation when the i.i.d.-set assumption is violated: under set heterogeneity σ_{het} up to 0.30, mimicry TPR stays 1.0 and the floor holds; only detection latency drifts up ($1.0 \to 1.7$ windows).

cells achieve $\leq$ 3-window detection with clean FPR $\leq 5\%$ and no missed episodes—a broad robust plateau around the chosen $(\tau{=}0.05, H{=}0.8)$. Detection degrades gracefully and predictably toward the plateau edges (small H raises false alarms; large τ raises misses), exactly as the Section V chain predicts.

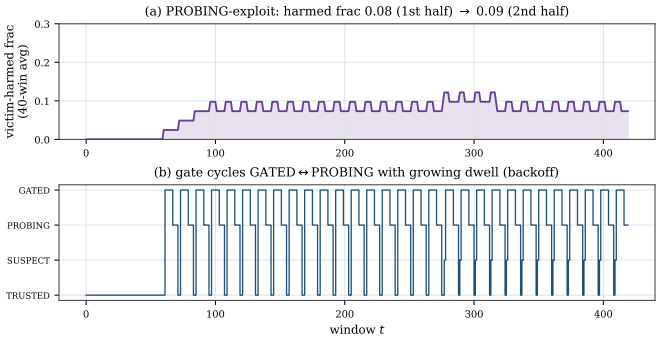
Misspecification robustness. Because the harness reward is built from the theorems' assumptions (bounded reward, near-i.i.d. sets), we test whether the results survive violating them. We give each set a fixed competence offset $\sim \mathcal{N}(0, \sigma_{het})$ (hot/cold sets, conflict skew), so Leader-C and Leader-F pools are exchangeable only in expectation—the β-bias regime of Proposition 1. Sweeping σ_{het} from 0 to 0.30 (Figure 10), mimicry detection stays at TPR 1.0 and clean FPR at 0, the steady floor holds (0.60–0.64%), and detection latency degrades only gently ($1.0 \to 1.7$ windows)—because secret *reseeding* re-randomizes which physical sets are leaders each epoch, averaging the heterogeneity bias down. The guarantees are thus not an artifact of the i.i.d. reward model.

G. Adaptive timing adversaries

Two threat-model-completeness attacks (Figure 11). **Boiling-frog** (Figure 11(a)): a very slow, stealthy degradation tuned to keep the per-window change tiny. It cannot evade—a one-sided CUSUM integrates the *level* $(K - \hat{\Delta})$, not the slope, so once $\hat{\Delta} < K$ it accumulates regardless of how slowly it arrived. Bouncer gates 7 windows after the true $\Delta{<}\tau$ crossing (vs. 1 for an abrupt attack); the slow descent



(a)



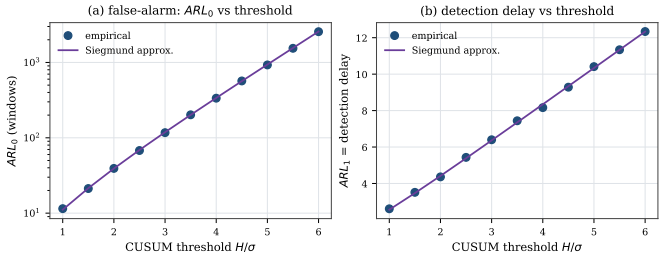
(b)

Fig. 11: Adaptive timing adversaries. (a) boiling-frog is still gated (a CUSUM integrates level, not slope); (b) the PROBING-exploit attacker is bounded to 8.6% harmed windows by measured re-trust + dwell backoff.

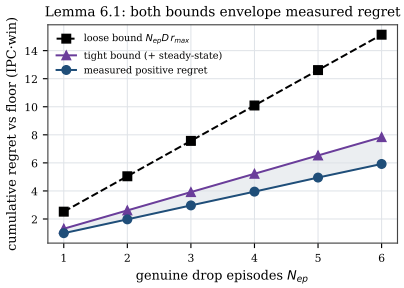
only *delays* detection, and the steady floor still holds (0.67%). **PROBING-exploit** (Figure 11(b)): a closed-loop attacker that watches the gate and backs off during GATED/PROBING to pass the audit, resuming when TRUSTED—the "wait out the cooldown" strategy. Because re-trust is *measured* (PROBING audits $\hat{\Delta}$) and the dwell backs off exponentially, the victim is harmed in only 8.6% of windows and the attacker spends $\sim 11\times$ more windows idle than effective; there is no fixed cooldown to wait out.

H. Theory validation

Figure 12(a) checks the worked CUSUM constants on an i.i.d. Gaussian signal across a threshold sweep: the empirical ARL_0 and detection delay match Siegmund's approximation eq. (5) (geometric-mean ratio 0.998, e.g. ARL_0 empirical 927 vs. theory 938 at $H{=}5\sigma$; a small H-dependent bias remains). This sweep is a *formula sanity check* covering the small-drift, heavy-overshoot regime where the 1.166 correction matters; the deployed detector runs in the opposite, strong-drift regime ($(K - \Delta)/\sigma_\Delta \approx 28$ at $n_L{=}32$), where the delay collapses to the deterministic limit $D \approx H/(K - \Delta)$ and ARL_0 is astronomically large (so $\alpha \approx 0$). Figure 12(b) overlays both the *loose* Lemma 1 bound $N_{ep} D r_{\max}$ and a *tight* version that charges the realized gap $q_0 - \mu_C^{\mathrm{att}}$—for D pre-detection



(a)



(b)

Fig. 12: Theory. (a) ARL matches Siegmund; (b) the safety-floor bound envelopes measured regret.

windows of full-resource degradation plus the steady-state n_L/n_{sets} Leader-L cost over the rest of the attack—on the measured worst-case cumulative regret ($\alpha T c_{sw} \approx 0$ here). Both envelope the measurement at N_{ep}=6: loose 15.1, tight 7.8, measured 5.9 IPC·win; the tight bound hugs the data while remaining an upper bound.

I. Real-systems integration (ChampSim)

To show the auditor is genuinely deployable, we built it as a real **ChampSim L1D prefetcher module**—the same set-dueling estimator, CUSUM, and gate FSM C++ as Section XII—with the "learned controller" a per-IP stride prefetcher, the fallback prefetch-*off*, the reward the per-set cache-hit rate, and the attack a cache-polluting corruption (the dueling "sets" here are virtual block-address buckets, $\mathrm{blk} \bmod 2048$, not physical L1D sets). It compiles and runs on SPEC CPU2017 with **no added latency on the cache-access critical path**: the confirmer/CUSUM/reseed/FSM are off-path and epoch-grained, and the per-access work is a 2-bit pool-tag lookup and an output mux—the same kind of table read DIP/DRRIP already carry. Because the real $\hat{\Delta}$ is far noisier than the synthetic one (only $m \approx 19$ accesses/bucket/window land, so $\hat{\Delta}$ swings in $[-0.8, +0.7]$), we ran at a looser operating point (τ=0, H=0.25) than the synthetic (τ=0.05, H=0.8, m=64); we disclose this rather than claim the synthetic R^2=0.997 estimator fidelity transfers (it does not—that is a controlled-harness result).

Across a three-trace suite (Table II, Figure 13) the candid lesson is: the auditor is only as good as its competence reward. The *floor cap is reliable*—on every trace Bouncer ends at the prefetch-off floor, so the corruption attack's worst

TABLE II: ChampSim L1D suite (9M-insn SPEC). The floor cap bounds the attack on every trace; the cache-hit reward proxy sets the clean tax.

SPEC trace	prefetch benefit	attack loss (unguard)	Bouncer clean tax
600.perlbench (compute)	+1.7%	0.3%	1.7%
619.lbm (streaming)	≈0 (hit-rate)	7.9%	2.3%
654.roms (mem-bound)	+12.7%	5.2%	11.2%

case is bounded: on streaming 619.lbm a corrupted prefetcher costs the unguarded config 7.9% IPC while Bouncer (already at the floor) is unharmed; on compute-bound 600.perlbench the attack is nearly harmless (0.3%) and so is the cap. But the per-set cache-hit reward *proxy* mis-estimates prefetch competence, so the clean tax ranges 1.7–11%: on memory-bound 654.roms stride prefetching genuinely helps IPC (1.14 vs the 1.01 off floor), yet the hit-rate signal misses that benefit (it comes from latency/MLP, not raw L1D hit count), so Bouncer over-gates. Two further honesty points: the gate is *competence*-driven, not attack-label-driven—on lbm it engaged before the attack onset, so this is a floor cap, *not* attack-specific detection; and the TRUSTED→GATED transition *dynamics* are shown where competence is controllable (the synthetic Figure 4), since no available naive-prefetcher reward exhibits a clean pre-attack TRUSTED state here.

It is tempting to blame the hit-rate proxy and propose the controller's *own* reward as the fix—but we tested exactly that and it is *insufficient*. Holding $(\tau, H, \text{window}, \text{partition}, \text{FSM})$ fixed and swapping *only* the reward, from per-set cache-hit to the prefetcher's own accuracy×timeliness, moves the roms clean tax only 11.2% → 9.7% and still loses 5% to the attacked-but-ungated prefetcher. The reason is a **fundamental tension in what set-dueling can measure**: the cache-hit reward is *set-local* (attributable to the dueling set) but a poor utility proxy, whereas the own-accuracy reward is a faithful proxy but *not set-local*—a prefetch triggered on a Leader-L set targets a *different* address that lands on a *different* set, so the dueling mis-credits it and $\hat{\Delta}$ collapses to ≈ 0. The dominant lever is therefore the observation window / signal SNR, not reward fidelity: lengthening the epoch (window 40k → 200k) removes the over-gating even with the crude proxy. This is a genuine limitation of set-dueling *for prefetchers*, whose benefit is inherently de-localized; *cache replacement*—where the decision and its hit/miss outcome share a set—is the clean case (the paper's "home turf" claim, now with a mechanism). The milestone this proof-of-deployment sets up is accordingly sharper: a long-epoch, *set-local* competence signal (latency-weighted, replacement-style), not merely "the controller's own reward."

The same auditor C++ also compiles and runs as a real ChampSim *LLC replacement* module (SRRIP-vs-LRU dueling on physical sets, hit/miss reward), confirming the mechanism is controller-agnostic in a real simulator; but at feasible (tens-of-millions-of-instruction) run lengths the per-window LLC dueling signal is sample-starved ($\sim$ 25 accesses/leader/window, $\hat{\Delta} \approx 0$), which is why the controlled

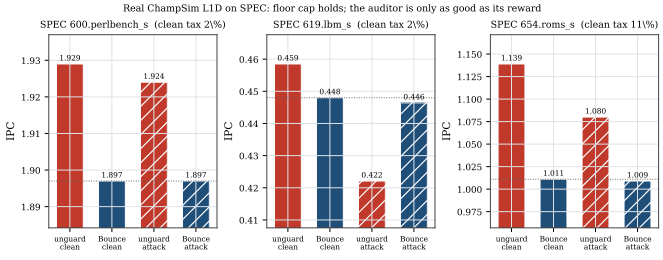


Fig. 13: Bouncer as a real ChampSim L1D prefetcher across a SPEC suite. On every trace Bouncer ends at the prefetch-off floor (the cap), bounding the corruption attack's worst case (lbm: 7.9% unguarded loss cannot reach the gated victim). The per-set cache-hit reward proxy sets the clean tax (1.7–11%): roms shows the failure mode—a genuinely-helpful prefetcher under-credited and over-gated. Swapping to the controller's own reward does *not* fix this (only 11.2% $\rightarrow$ 9.7%); a prefetcher's benefit is not set-local, so set-dueling needs a longer epoch or a set-local signal (replacement is the clean case).

TABLE III: Overhead budget (analytical).

Component	Storage	Per-decision work
Set-dueling counters	64 B (reuse ATD)	reuse existing
S_{in} RP + quantile sketch	224 B	$\leq$ few adds on $1/k$
S_{res} forward model g	32 B	16 MACs on $1/k$
CUSUM detectors ($\times 4$)	16 B	2 add/cmp each
Total	**$\sim$336 B**	off critical path

competence and transition results live in the synthetic harness. Both real instantiations point to the same milestone: long-horizon, DIP-style accumulation with the controller's native reward.

XII. Overhead Budget

Table III is the *analytical* overhead budget (an RTL-level measurement is future work). Set-dueling counters reuse the existing ATD/sampler; the Tier-A sketches and the 16-weight forward model sit in adjacent SRAM and update on a sampled $1/k$ of decisions; the CUSUM detectors are two registers and a comparator each. Total $\sim$ 336 bytes, $\sim$ 0.13% of a 256 KB SRAM, an estimated $<1\%$ energy, and—because the confirmer is sampled, off-path, and epoch-grained—**no added latency on the cache- access critical path**, the non-negotiable constraint. The only per-access work is the 2-bit pool-tag lookup and output mux (a table read comparable to existing DIP/DRRIP set-dueling), not the estimator or gate logic.

XIII. Related Work

Learned microarchitectural controllers. A growing body of work replaces hand-tuned heuristics with online or offline-trained predictors across every major structure Bouncer targets. RL memory schedulers [4] and RL prefetchers [1] learn performance-coupled policies online; learning-based cache replacement imitates Belady's optimum with PC-indexed predictors [2], distilled neural models [3], or signature-based hit predictors [20]; and neural and geometric-history branch predictors [5], [6] have become the dominant adaptive prediction substrate. These designs share a failure surface: their advantage over a simple baseline (LRU/RRIP, a stride prefetcher, a bimodal predictor) is workload- and phase-dependent and can silently invert under distribution shift. Bouncer is agnostic to which of these controllers it wraps; rather than proposing a new policy, it provides the missing runtime guarantee that any of them never costs more than its own known-safe fallback.

Set dueling and adaptive policy selection. Set dueling [14] dedicates a handful of sampled sets to each of two competing policies and lets a single saturating counter steer the followers, a mechanism extended to re-reference prediction [15], shared-cache thread-aware insertion [21], and evicted-address filtering [22], and rooted in the sampling-monitor methodology of utility-based partitioning [23] and sampling dead-block prediction [24]; a recent retrospective documents its industrial adoption and limits [25]. Bouncer borrows the dedicated-sets-plus-counter substrate but repurposes it from policy *selection* to policy *trust*: where DRRIP duels two heuristics to pick the empirical winner, Bouncer duels the learned controller against its fallback to *certify* a competence advantage and, crucially, keeps the run-C/run-fallback assignment *secret*. Classic set dueling assumes cooperative policies and is content to follow whichever wins; Bouncer must instead defend against a controller that may be steered or drifting, which is why its dueling sets feed a change detector and a trust gate rather than a preference counter.

Safe RL, runtime assurance, and Simplex. The closest architectural ancestor is Simplex [26]: a simple, formally verified safety controller bounds an unverifiable high-performance one via a decision module. Shielding [27] synthesizes a correct-by-construction shield from a temporal-logic specification, and constrained- and Lyapunov-based safe RL [28], [29], [30], [31] keep policies inside a certified-safe set, with safe policy improvement [32], [33] reverting to the data-collecting baseline wherever improvement is not statistically supported (surveyed in [34]). Bouncer inherits the "use simplicity to control complexity" posture and the bootstrap-to-baseline reflex, but departs in what it must prove. Simplex and shielding require a *verified model of the safety envelope*; constrained RL requires a known cost function. Bouncer needs neither: microarchitectural "safety" is not a state-space invariant but a relative performance floor, so it certifies only that the learned controller's *realized reward* beats the fallback's on randomized dueling sets, with a bounded-regret safety floor (our Lemma) that holds without any reachability proof or verified plant model.

Change detection and OOD monitoring. Bouncer's Tier-B confirmer runs a one-sided CUSUM [16] over its competence signal, inheriting the quickest-detection theory that makes CUSUM minimax-optimal for detection delay under a false-alarm constraint [18], [19], with ARL-to-threshold sizing from sequential analysis [17], [35]. Tier-A tripwires draw on the complementary literature of input-side monitoring: softmax-confidence OOD baselines [11], conformal validity under

exchangeability [36], [37], concept-drift detection [13], and representation-level two-sample shift tests [12]. The essential distinction is what is measured. OOD and drift monitors flag when the *input distribution* moves, which is necessary but not sufficient and, fatally, *mimickable*: an adversary can keep covariate statistics in-distribution while degrading control quality. Bouncer instead measures *realized competence* through secret dueling, so the trigger is the controller's actual reward deficit, not a proxy for it. This is why Bouncer cheaply uses input monitors only as a first-tier gate and reserves its trust decision for the competence statistic.

Microarchitectural security and co-tenant attacks. The controllers Bouncer audits are also attack surfaces. Contention and timing channels on caches [38], [39], [10], [40] and occupancy channels [9] leak across cores and VMs in shared clouds [41], while branch predictors [42], [8] and prefetchers [43], [7] expose secret adaptive state that an attacker can read out or steer. A learned controller widens this surface: its policy can be poisoned into amplifying leakage or into co-tenant denial of service. Prior defenses harden specific channels; Bouncer is orthogonal and complementary, bounding the *aggregate* damage any steered controller can do by capping its cost at the vetted fallback. Its mimicry resistance (our Proposition) follows precisely from the secrecy of the set assignment, so an attacker who can observe and game stealthy counter-based detectors [40] still cannot tell which accesses are being scored.

ML-for-systems robustness. Deployed learned components are brittle in exactly the regimes Bouncer guards. Oracle-imitation cache controllers lose accuracy off-distribution [44], learned indexes degrade on irregular data [45], and the Park benchmark documents why RL-for-systems policies behave unlike RL-for-games under noisy, non-stationary workloads [46]; this is the hidden, system-level risk catalogued in [47]. Production systems respond with bespoke guardrails: SLO-clamped colocation [48], OOM-bounded autoscaling [49], and fault-triggered fallback in learned software [50]. Bouncer generalizes these one-off, fault- or threshold-triggered safeguards into a single hardware-cheap mechanism with a *continuous* competence audit and a provable performance floor, applicable to any learned microarchitectural controller rather than one tuned guardrail per deployment.

XIV. Discussion and limitations

Counterfactual cost. Controllers without clean set-sampling (the memory scheduler) force a model-based $\hat{\Delta}$, which weakens Proposition 1; we scope sampling-friendly controllers first. **Redistributive mimicry and the coverage hole.** A global $\hat{\Delta}$ cannot see an attack that preserves the global mean; per-domain dueling raises detection but does *not* fully close it—a finite leader budget B has a resolution floor $\delta_R^{\min}(B)$, and a slow-bleed adversary that keeps its per-window harm below it evades indefinitely (Proposition 1). We therefore claim mimicry resistance only for declared domains audited at that resolution with an intact secret, and name the sub- resolution coverage hole as an open vulnerability rather than hiding it. **Ground-truth labeling.** "Off-policy" is operationalized as a sustained competence drop beyond a fixed margin, against which we measure latency. **Overhead vs. sensitivity.** The auditor dies if the Tier-B duty cycle is too high; the minimal-overhead operating point—the knee of Figure 6(a)—is reported as the result, not a footnote. **From simulation to silicon.** The controlled quantitative claims are validated on a faithful harness; the real ChampSim L1D module (Section XI-I) shows the mechanism survives a real simulator and bounds a corrupted prefetcher to the safe floor across a 3-trace SPEC suite—a full SPEC/GAP sweep with a timeliness-aware reward (and the production controllers of Table I) is the natural next step. **Reward *localizability*, not just fidelity, is decisive.** The ChampSim runs make this concrete and correct a tempting but wrong fix. A per-set cache-hit reward is a poor proxy for a prefetcher's true (latency/MLP) benefit and over-gates a helpful prefetcher on roms (11% tax)—but we showed (Section XI-I) that swapping to the controller's *own* accuracy$\times$timeliness reward does *not* fix it ($11.2\% \to 9.7\%$ at fixed window). Set-dueling needs a *set-local* reward, and a prefetcher's benefit is intrinsically de-localized (it fetches a different set than it was triggered on), so the faithful reward is the one that cannot be dualized. The practical levers are a longer epoch (which removes the over-gating empirically) and a set-local signal; the structurally clean case is a controller whose decision and outcome share a set—*replacement*. This is a property of the controller-reward *geometry*, and it sharply scopes where set-dueling competence auditing is faithful.

XV. Conclusion

Bouncer turns "is the learned controller still earning its keep?" into one measurable scalar and bounds the answer's cost. By repurposing the set-dueling substrate into a *secret* competence audit, gating an expensive confirmer behind cheap tripwires, and proving a safety floor tied directly to the CUSUM constants, it caps the worst-case behavior of any learned microarchitectural controller at its vetted fallback—under benign drift and an adaptive mimicry adversary alike—for ~ 336 bytes and zero added datapath latency. The robustness is not free: we show it is purchased exactly by the secrecy of the dueling sets, and we quantify the price of losing it.

Artifact Appendix

The full artifact is released: the auditor and environment (`bouncer/`), one script per evaluation phase (`experiments/`), all result JSON and figures, the real ChampSim L1D module (`champsim_plugin/bouncer_pref/`), and the paper source. `./run_all.sh` regenerates every synthetic result, figure, and the PDF in $\sim$4 minutes on a laptop (Python 3.12, numpy/scipy/matplotlib); all RNG is explicitly seeded, so results are deterministic. `champsim_plugin/setup_champsim.sh` clones and builds ChampSim, installs the Bouncer prefetcher module, fetches a public SPEC CPU2017 trace, and reproduces

the Section XI-I safety-floor study (C++17 toolchain + vcpkg). The standalone auditor additionally builds with `c++ -std=c++17 bouncer.cc -o bouncer_selftest`.

REFERENCES

[1] R. Bera, K. Kanellopoulos, A. V. Nori, T. Shahroodi, S. Subramoney, and O. Mutlu, "Pythia: A customizable hardware prefetching framework using online reinforcement learning," in *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2021, pp. 1121–1137.

[2] A. Jain and C. Lin, "Back to the future: Leveraging belady's algorithm for improved cache replacement," in *Proceedings of the 43rd International Symposium on Computer Architecture (ISCA)*, 2016, pp. 78–89.

[3] Z. Shi, X. Huang, A. Jain, and C. Lin, "Applying deep learning to the cache replacement problem," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2019, pp. 413–425.

[4] E. Ipek, O. Mutlu, J. F. Martínez, and R. Caruana, "Self-optimizing memory controllers: A reinforcement learning approach," in *Proceedings of the 35th Annual International Symposium on Computer Architecture (ISCA)*, 2008, pp. 39–50.

[5] D. A. Jiménez and C. Lin, "Dynamic branch prediction with perceptrons," in *Proceedings of the 7th International Symposium on High-Performance Computer Architecture (HPCA)*, 2001, pp. 197–206.

[6] A. Seznec and P. Michaud, "A case for (partially) TAgged GEometric history length branch prediction," *Journal of Instruction-Level Parallelism (JILP)*, vol. 8, pp. 1–23, 2006.

[7] Y. Guo, A. Zigerelli, Y. Zhang, and J. Yang, "Adversarial prefetch: New cross-core cache side channel attacks," in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022, pp. 1458–1473.

[8] D. Evtyushkin, R. Riley, N. B. Abu-Ghazaleh, and D. Ponomarev, "BranchScope: A new side-channel attack on directional branch predictor," in *Proceedings of the 23rd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2018, pp. 693–707.

[9] A. Shusterman, L. Kang, Y. Haskal, Y. Meltser, P. Mittal, Y. Oren, and Y. Yarom, "Robust website fingerprinting through the cache occupancy channel," in *28th USENIX Security Symposium (USENIX Security 19)*. Santa Clara, CA: USENIX Association, 2019, pp. 639–656.

[10] F. Liu, Y. Yarom, Q. Ge, G. Heiser, and R. B. Lee, "Last-level cache side-channel attacks are practical," in *2015 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 2015, pp. 605–622.

[11] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," in *International Conference on Learning Representations (ICLR)*, 2017.

[12] S. Rabanser, S. Günnemann, and Z. C. Lipton, "Failing loudly: An empirical study of methods for detecting dataset shift," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[13] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 44:1–44:37, 2014.

[14] M. K. Qureshi, A. Jaleel, Y. N. Patt, S. C. Steely, and J. Emer, "Adaptive insertion policies for high performance caching," in *Proceedings of the 34th Annual International Symposium on Computer Architecture (ISCA)*. ACM, 2007, pp. 381–391.

[15] A. Jaleel, K. B. Theobald, S. C. Steely, and J. Emer, "High performance cache replacement using re-reference interval prediction (rrip)," in *Proceedings of the 37th Annual International Symposium on Computer Architecture (ISCA)*. ACM, 2010, pp. 60–71.

[16] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.

[17] D. Siegmund, *Sequential Analysis: Tests and Confidence Intervals*, ser. Springer Series in Statistics. New York: Springer-Verlag, 1985.

[18] G. Lorden, "Procedures for reacting to a change in distribution," *The Annals of Mathematical Statistics*, vol. 42, no. 6, pp. 1897–1908, 1971.

[19] G. V. Moustakides, "Optimal stopping times for detecting changes in distributions," *The Annals of Statistics*, vol. 14, no. 4, pp. 1379–1387, 1986.

[20] C.-J. Wu, A. Jaleel, W. Hasenplaugh, M. Martonosi, S. C. Steely, and J. Emer, "SHiP: Signature-based hit predictor for high performance caching," in *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. ACM, 2011, pp. 430–441.

[21] A. Jaleel, W. Hasenplaugh, M. K. Qureshi, J. Sebot, S. C. Steely, and J. Emer, "Adaptive insertion policies for managing shared caches," in *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques (PACT)*. ACM, 2008, pp. 208–219.

[22] V. Seshadri, O. Mutlu, M. A. Kozuch, and T. C. Mowry, "The evicted-address filter: A unified mechanism to address both cache pollution and thrashing," in *Proceedings of the 21st International Conference on Parallel Architectures and Compilation Techniques (PACT)*. ACM, 2012, pp. 355–366.

[23] M. K. Qureshi and Y. N. Patt, "Utility-based cache partitioning: A low-overhead, high-performance, runtime mechanism to partition shared caches," in *Proceedings of the 39th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE Computer Society, 2006, pp. 423–432.

[24] S. M. Khan, Y. Tian, and D. A. Jiménez, "Sampling dead block prediction for last-level caches," in *Proceedings of the 43rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE Computer Society, 2010, pp. 175–186.

[25] M. K. Qureshi, A. Jaleel, Y. N. Patt, S. C. Steely, and J. Emer, "Retrospective: Adaptive insertion policies for high-performance caching," in *ISCA@50 25-Year Retrospective: 1996–2020*, 2023, retrospective on the ISCA 2007 DIP/set-dueling paper.

[26] L. Sha, "Using simplicity to control complexity," *IEEE Software*, vol. 18, no. 4, pp. 20–28, 2001.

[27] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI)*. AAAI Press, 2018, pp. 2669–2678.

[28] E. Altman, *Constrained Markov Decision Processes*. Chapman and Hall/CRC, 1999.

[29] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained policy optimization," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, D. Precup and Y. W. Teh, Eds., vol. 70. PMLR, 2017, pp. 22–31.

[30] Y. Chow, O. Nachum, E. Duenez-Guzman, and M. Ghavamzadeh, "A lyapunov-based approach to safe reinforcement learning," in *Advances in Neural Information Processing Systems 31 (NeurIPS)*, 2018, pp. 8092–8101.

[31] F. Berkenkamp, M. Turchetta, A. P. Schoellig, and A. Krause, "Safe model-based reinforcement learning with stability guarantees," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 908–918.

[32] P. S. Thomas, G. Theocharous, and M. Ghavamzadeh, "High confidence policy improvement," in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 37. PMLR, 2015, pp. 2380–2388.

[33] R. Laroche, P. Trichelair, and R. T. des Combes, "Safe policy improvement with baseline bootstrapping," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 97. PMLR, 2019, pp. 3652–3661.

[34] J. García and F. Fernández, "A comprehensive survey on safe reinforcement learning," *Journal of Machine Learning Research*, vol. 16, pp. 1437–1480, 2015.

[35] A. Tartakovsky, I. Nikiforov, and M. Basseville, *Sequential Analysis: Hypothesis Testing and Changepoint Detection*, ser. Monographs on Statistics and Applied Probability. Boca Raton, FL: Chapman & Hall/CRC, 2014.

[36] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. New York: Springer, 2005.

[37] J. Lei, M. G'Sell, A. Rinaldo, R. J. Tibshirani, and L. Wasserman, "Distribution-free predictive inference for regression," *Journal of the American Statistical Association*, vol. 113, no. 523, pp. 1094–1111, 2018.

[38] D. A. Osvik, A. Shamir, and E. Tromer, "Cache attacks and countermeasures: The case of AES," in *Topics in Cryptology – CT-RSA 2006, The Cryptographers' Track at the RSA Conference*, ser. Lecture Notes in Computer Science, vol. 3860. Springer, 2006, pp. 1–20.

[39] Y. Yarom and K. Falkner, "FLUSH+RELOAD: A high resolution, low noise, L3 cache side-channel attack," in *23rd USENIX Security Symposium (USENIX Security 14)*. San Diego, CA: USENIX Association, 2014, pp. 719–732.

[40] D. Gruss, C. Maurice, K. Wagner, and S. Mangard, "Flush+flush: A fast and stealthy cache attack," in *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, ser. Lecture Notes in Computer Science, vol. 9721. Springer, 2016, pp. 279–299.

[41] T. Ristenpart, E. Tromer, H. Shacham, and S. Savage, "Hey, you, get off of my cloud: Exploring information leakage in third-party compute clouds," in *Proceedings of the 16th ACM Conference on Computer and Communications Security (CCS)*. ACM, 2009, pp. 199–212.

[42] D. Evtyushkin, D. Ponomarev, and N. Abu-Ghazaleh, "Jump over ASLR: Attacking branch predictors to bypass ASLR," in *49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2016, pp. 1–13.

[43] D. Gruss, C. Maurice, A. Fogh, M. Lipp, and S. Mangard, "Prefetch side-channel attacks: Bypassing SMAP and kernel ASLR," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2016, pp. 368–379.

[44] E. Z. Liu, M. Hashemi, K. Swersky, P. Ranganathan, and J. Ahn, "An imitation learning approach for cache replacement," in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020, pp. 6237–6247.

[45] R. Marcus, A. Kipf, A. van Renen, M. Stoian, S. Misra, A. Kemper, T. Neumann, and T. Kraska, "Benchmarking learned indexes," *Proceedings of the VLDB Endowment (PVLDB)*, vol. 14, no. 1, pp. 1–13, 2020.

[46] H. Mao, P. Negi, A. Narayan, H. Wang, J. Yang, H. Wang, R. Marcus, R. Addanki, M. K. Shirkoohi, S. He, V. Nathan, F. Cangialosi, S. B. Venkatakrishnan, W.-H. Weng, S. Han, T. Kraska, and M. Alizadeh, "Park: An open platform for learning-augmented computer systems," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019, pp. 2490–2502.

[47] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems 28 (NeurIPS)*, 2015, pp. 2503–2511.

[48] D. Lo, L. Cheng, R. Govindaraju, P. Ranganathan, and C. Kozyrakis, "Heracles: Improving resource efficiency at scale," in *Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA)*, 2015, pp. 450–462.

[49] K. Rzadca, P. Findeisen, J. Swiderski, P. Zych, P. Broniek, J. Kusmierek, P. Nowak, B. Strack, P. Witusowski, S. Hand, and J. Wilkes, "Autopilot: Workload autoscaling at Google," in *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys)*, 2020, pp. 16:1–16:16.

[50] V. Carbune, T. Coppey, A. Daryin, T. Deselaers, N. Sarda, and J. Yagnik, "Smartchoices: Augmenting software with learned implementations," *arXiv preprint arXiv:2304.13033*, 2023.